

NetMamba+: what we built and measured

Customer discussion · Tuesday, 15 September 2026

We trained and tested the authors' original NetMamba+ classifier on compatible CICIoT2022 flows using a documented NVIDIA GB10 compatibility port. Three runs completed the full source-based fine-tuning schedule. The repository contains the code, measured results, a saved-inference replay, an editable slide deck and the research record needed to inspect the work.

Measured mean test accuracy: 86.65% across seeds 0, 1 and 2. Mean weighted F1 is 86.67% and mean macro F1 is 87.27%. Every seed is reported. The fixed test set has 1,041 flows; the selected classifier for the demo is seed 0, chosen before test scores.

This is a source-based reproduction attempt on a new runtime. It does not establish the paper's full pretraining history, exact paper-protocol accuracy, a live production IDS, or NPU/SmartNIC execution. Paper Table IV reports NetMamba+ CICIoT2022 accuracy and F1 of 97.50%; that is the authors' result under different conditions.

What was actually executed

- Full metadata scans of both supplied packet CSVs: 1,410,255 CICIDS2017 rows and 79,881 UNSW rows.
- Validation of all 10,404 released CICIoT2022 flows and their six-class mapping.
- Native CUDA extension builds and seven forward/backward numerical checks, repeated in a second isolated environment.
- A masked-pretraining functional run: 132 updates on training-only flows, finite loss and a saved checkpoint.
- Three full fine-tuning runs: 120 epochs and 7,920 updates each, with validation-selected saved classifiers.
- Fresh strict classifier inference, independent metrics, a browser replay, and separately scoped latency measurements.

The useful AI-assisted research is available as a reviewed research record. Claims are tied to commands, hashes, logs and predictions rather than model-generated explanations alone.

Evidence: [customer index and downloads](#), [research record](#).

What goes in, and what comes out

One model example is a **flow**, meaning packets grouped by an established connection rule and ordering. The two uploaded CSVs describe individual packet payloads and omit the identity and order needed to reconstruct the required flow sequence. Five adjacent rows cannot be assumed to be one connection. A classifier trained directly on those CSVs would be an adaptation.

The actual experiment uses the authors' separate six-class CICIoT2022 flow release. The original loader takes:

Input view	Native representation	What the model receives
Byte content	First 5 packets × 320 stored bytes	Normalized tensor [B, 1, 1, 1600]; B is batch size
Packet sizes	First 20 values	Clipped and padded size sequence
Arrival intervals	First 20 values	Transformed and padded timing sequence

Four-byte strides create 400 byte tokens. Twenty size tokens, twenty timing tokens and three learned summary tokens make **443 tokens per complete classifier input**. Four unidirectional Mamba blocks process that sequence with 256-dimensional embeddings. Each block updates a learned state as it scans the feature tokens; this internal state is not a live connection cache shared across flows.

The model sums the three modality summary vectors and applies a six-class head. It outputs six unnormalized scores, called logits. The largest score determines a label. Softmax turns scores into a convenient display distribution, but those values are not calibrated probabilities of malicious traffic.

Index	Released class meaning
0 / 1	Flood / RTSP Brute Force
2 / 3	IoT Power-Audio / Power-Other
4 / 5	IoT Power-Cameras / Power-Home Automation

The benchmark label is used to train or evaluate the model, not as an inference feature. Six output shapes alone do not prove class meanings: the repo binds each saved classifier's SHA-256 to the class mapping and checks that mapping during replay.

Sources: [paper v1](#), [pages 8–11](#), [pinned native loader and model source](#).

How training and inference work

1. Masked pretraining learns a representation

The model hides parts of the byte, size and timing inputs and learns to reconstruct them. The selected source settings mask 90% of byte strides and 15% of each sequence modality. Reconstruction decoders provide the training signal without using attack-category targets.

The authors' released pretraining checkpoint contains reconstruction parameters and no classifier head. Its compatible encoder weights initialize the main fine-tuning runs. The exact original Browser/Kitsune corpus and complete training history remain unestablished. Separately, our functional test on CICIOT2022 training data completed two epochs and 132 updates, with epoch-average loss falling from about 9.52 to 6.13. This validates the pretraining path, not the paper's full pretraining experiment.

2. Fine-tuning learns the labeled categories

The original training code adds the six-class head and updates it along with the encoder using labeled flows. After each epoch, validation accuracy determines whether to replace the saved best checkpoint. The final test pass uses that validation-selected checkpoint. It does not select the best test epoch.

Setting	Measured source-based experiment	Paper v1
Fine-tuning budget	120 epochs per seed	120 epochs
Batch / accumulation / processes	128 / 1 / 1	Batch 64; paragraph differs
Learning rate	blr 0.002 → effective maximum 0.001	Reported 0.002
Runtime	GB10, Python 3.12, Torch 2.9.1/cu130	A100 setup, Torch 2.1.1
Main initialization	Released pretraining checkpoint	Paper's pretraining history

3. Inference freezes the selected classifier

The separate evaluator loads all classifier weights with strict shape/key checks, verifies class meanings, runs the original tensor loader and uses the native evaluation engine. That engine uses CUDA autocast even though the fine-tuning preset disables AMP for training. Fresh-process inference verifies the saved model, not just the in-memory training object.

The runtime port builds the authors' Mamba fork with recorded CUDA 13 compatibility edits. Seven numerical checks compared optimized forward/backward operations with reference paths before accepting the runtime. The original model and loader checkout remains unmodified.

Evidence: [runbook](#), [runtime and build evidence](#).

What the results establish

Seed	Accuracy	Weighted F1	Macro F1
0	91.26%	91.26%	91.58%
1	84.05%	84.08%	84.86%
2	84.63%	84.67%	85.38%
Mean	86.65%	86.67%	87.27%

Accuracy ranges from 84.05% to 91.26%, with a sample standard deviation of 4.00 percentage points. The mean is 10.85 points below the paper's 97.50% reference. These differences are visible; their causes have not been isolated by controlled experiments.

Accuracy counts correct predictions. Weighted F1 weights classes by true support; macro F1 weights them equally. Complete confusion matrices are retained. The smallest class has only 41 test examples.

Native final-test results, fresh strict evaluation and independently recomputed metrics agree. Checks confirmed complete training, finite and changed weights, validation selection and optimizer counters. Model-only exports preserve the selected tensors exactly.

The official split has 8,323 training / 1,040 validation / 1,041 test flows. Five exact stored inputs overlap train/validation, and six overlap train/test. Recorded capture identifiers do not overlap, but physical capture independence and prior checkpoint exposure remain unknown.

Repeated tests verify the same frozen result; they did not guide tuning. Three seeds on one split do not establish performance on customer networks.

Model-only inference timing

Batch	Median latency	95th percentile	Flows/s from mean
1	0.88 ms	0.93 ms	1,120.0
16	4.32 ms	4.64 ms	3,679.1
128	36.55 ms	37.38 ms	3,502.3

GB10, seed 0, batches 1/16/128, 20 warmups and 100 CUDA-synchronized repetitions each, after training finished. Capture, flow waiting, preprocessing, loading, transfers and alerts are excluded. These are not network line-rate measurements.

Paper Table IV reports 97.50% accuracy/F1 under different conditions. Its separate online figures describe byte-only NetMamba, not this measurement.

Evidence: [complete measured results and hashes](#).

Present a working demo and explain the next build

The delivered HTML replay shows **actual predictions recorded from the selected classifier**. It opens without a GPU or network connection. The repo also contains commands to regenerate the predictions through real GPU inference.

1. Open the replay and press Play replay.
2. Explain the input: byte content, packet sizes and timing for one flow.
3. Press Show all, then select Prediction errors. Walk through an actual wrong prediction against its dataset label.
4. Show the classifier fingerprint and the full test metrics. A display confidence score is not an operational attack probability.
5. Select Attack-class predictions. Explain that Flood and RTSP Brute Force are the two attack labels in this benchmark.

Playback pace changes the display only. This page does not capture or block packets, and it does not implement a production alert policy. The recorded walkthrough and PDF are meeting backups; the underlying inference run and its evidence remain available.

A simple real IDS could connect these components

Mirrored traffic → capture and flow cache → compatible feature extraction → model queue → saved classifier → reviewed alert policy and event log.

The measured classifier is one component. The first integration milestone is a reviewed PCAP-to-tensor comparison that establishes grouping, direction, ordering, header extraction, timing units and padding. Upstream extraction helpers currently need path changes and a missing common module; they are not a turnkey verified online adapter here.

Then run an alert-only trial on independent traffic. Measure false alerts per hour, missed attacks, capture loss, queue growth, active-flow memory and end-to-end delay. Traffic has to be observed before its five-packet/twenty-entry representation is available; that waiting cost is absent from the model-only timing.

The current six labels are not comprehensive attack coverage. Unknown traffic handling, calibration and an operational threshold require their own validation. The present evidence supports a research demo and an integration starting point.

Evidence: [demo and rehearsal instructions](#), [hardware/IDS roadmap](#).

How this could map to an NPU or SmartNIC

A SmartNIC can help steer and process packets; it does not automatically execute the neural classifier. A sensible initial split is NIC steering, host or DPU flow handling, GPU classification and software alert review. NVIDIA DOCA Flow is one candidate for the packet-handling side.

An AI NPU port needs a named device and compiler. The difficult operations include the input-dependent selective scan, causal convolution and fused normalization, currently implemented with custom CUDA/Triton code. The real Torch export probe reports **not captured in the tested Torch path**: graph capture cannot trace the custom causal-convolution CUDA call. Eager GPU inference succeeded first. This is not an NPU compiler or hardware test.

The 1,870,080 trainable parameters account for about 7.48 MB in FP32, before buffers, activations, recurrent state and workspaces. Fixed input shapes help sizing, but advertised TOPS alone cannot determine latency. Lower precision requires held-out per-class accuracy checks, not just successful conversion.

Current DOCA GPUNetIO documentation notes that DGX Spark does not support GPUDirect RDMA. A direct NIC-to-GPU path therefore cannot be assumed on this host. The exact NIC, interconnect, firmware, SDK, compiler and accelerator must be specified before performance commitments.

What remains missing

- Established Browser/Kitsune pretraining inputs and the released checkpoint's complete history.
- An exact paper-protocol reproduction across its datasets, baselines and ablations.
- A verified live capture-to-tensor adapter and independent customer-traffic evaluation.
- Calibrated confidence, unknown-attack handling and an operational alert/blocking policy.
- Actual NPU or SmartNIC model execution, quantization validation and on-target system measurements.
- Confirmed distribution and commercial-use terms for inherited third-party research assets.

The defensible statement: We trained and tested the original NetMamba+ classifier on the authors' compatible CICIOT2022 flows using a documented GB10 runtime port. We can show the saved-model predictions, exact measured results, errors and reproducible commands. The next step is an independently evaluated capture-to-alert integration on a specified deployment target.

Sources: [DOCA Flow](#), [DOCA GPUNetIO](#), [OpenVINO NPU](#), [all project evidence](#).

Set up, use and test it

View and present: only a browser is needed

Open the [plain-language companion](#). It follows the same numbered slides and speaker script, with everyday comparisons and a glossary. Open the replay, press Play replay and Show all, then inspect Prediction errors. You should see 1,041 total flows, 91 errors and 91.26% seed-0 accuracy.

For an offline meeting, extract the release ZIP and open docs/customer/demo/guide.html and index.html. PDFs, editable PowerPoint with notes, a text script and a recorded video are also included. The replay is recorded GPU inference; it does not watch or block your network.

Verify the package: Python 3.10 or newer

Clone or unzip the project. In the project root, run `python3 -m unittest discover -s tests -v`, then `python3 tools/verify_package.py`. These CPU checks were executed on Linux; Windows/macOS runs were not tested.

Expect Ran 54 tests and OK, followed by verifier JSON containing "status": "passed". No research asset download or GPU is needed for these checks. A mismatch must be investigated; regenerating hashes does not fix an unsupported claim.

Actual check	Recorded outcome
CPU tests	54 passed; harness and failure handling
Native GPU runtime	Seven numerical checks passed in each of two built environments
Three model-only exports	Fresh strict evaluations reproduced all three original scores
Full unlabeled inference	Seed-0 predicted labels and logits for all 1,041 flows matched the frozen record in this run
Public replay	Playback, filters and reset passed; no overflow at 320/390/768/1440 pixels
Torch graph capture	Custom causal-convolution export unsupported in the tested path; GPU inference still passed

Run new training or inference: follow the GB10 runbook

Fetch the pinned source and assets; build the GPU environment; run numerical checks; train a classifier; then use `evaluate.py`, `replay.py` or `predict.py`. Use fresh output directories and keep checkpoint provenance. Raw traffic and inherited weights are not in the ZIP.

Tested GPU profile: aarch64 Python 3.12.3, GB10, driver 580.126.09, CUDA toolkit 13.0.88. CPU checks do not repeat GPU training. Rechecking one test split does not create an independent holdout.

Commands and expected files: [simple quickstart](#), [full runbook](#), [dated verification](#).

The paper, the data and our changes

The paper studies intrusion detection: recognizing potentially suspicious traffic patterns. NetMamba+ combines bytes, packet sizes and time gaps using a Mamba sequence model. It is not a chatbot. The authors provide the research implementation; this repo adds a checked execution route and measured evidence.

The uploaded tables are not the training flow release

Data	What it contains	What we did with it
CICIDS2017 packet CSV	1,410,255 rows, 15 labels	Parsed all rows and profiled metadata
UNSW packet CSV	79,881 rows, 10 labels	Same compatibility investigation
Authors' CICIoT2022 flow JSON	10,404 flows, six classes	Actual model training and evaluation

Each CSV has 1,500 payload-byte slots plus TTL, length, protocol, time delta and label. These exports lack connection identity and established per-flow order. Neighboring rows cannot be assumed to form a flow. CICIDS2017 and CICIoT2022 are distinct datasets.

Compared with the authors' repository

Area	Original research	Our work / disclosed difference
Model and loader	Native NetMamba+ source	Original 103 tracked files unchanged; source pins verified
Build and hardware	Older extensions; paper A100 / Torch 2.1.1	GB10 / Torch 2.9.1; compiler edits only in disposable build copies
Fine-tuning settings	Paper batch 64, rate 0.002	Released source preset: batch 128, effective maximum rate 0.001
Evaluation and use	Native research scripts	Strict checkpoint evaluation, unlabeled prediction, class-mapping provenance and recorded replay
Evidence and explanation	Authors' reported results	Three complete measured runs, all predictions/errors, tests, setup guide, PDFs, slides and research record

Main training uses the released pretrained encoder. Full pretraining history, the cause of the accuracy gap, live capture, independent customer validation and target hardware execution remain unestablished.

More detail: [paper v1](#), [pinned authors' repository](#), [file and dataset comparison](#), [map of all seven customer questions](#).